Name: Anushka Suvarna
UID: 2023300241
Batch: B
Course Code: CS303
Experiment no.: 1
Title of the Experiment: Prolog Program using knowledge base and inference rules

Theory:
This experiment is based on the idea of using Prolog, a logic programming language, to build a car recommendation system. The knowledge about different cars is stored as facts in the form of brand, color, seating capacity, fuel economy, price, transmission type, and category (SUV or sedan). Using these facts, rules are created to check whether a car matches the preferences given by the user. Prolog then searches through the stored knowledge base and provides suitable options, making the system work like a smart assistant for car selection.

Procedure:

1. Theory:

Logic programming is a style of programming where the solution is obtained by writing facts and rules instead of step-by-step instructions. Facts represent real-world truths, for example: Toyota is a sedan, while rules describe conditions under which queries can be answered, for example: A car matches if it fits type, color, and budget. The system processes a user's input query and checks it against the facts and rules to return suitable results.

2.Design Steps:

- Define facts for cars (Brand, Colour, Seats, Fuel Economy, Price, Gear Type, Type).
- Take input from the user (type, colour, budget, transmission).
- Write rules to check which cars fit the user's requirements.
- If no exact match is found, suggest alternatives within 20% more budget.
- Display the car details.

3.Program Code:

```
% Car Database

% Car Facts (Brand, Color, Seats, FuelEconomy, Price, GearType, Type)

car("Mercedes","black",7,"good",5000,"automatic","sedan").

car("BMW","red",4,"medium",2000,"manual","suv").
```

```
car("Toyota","white",5,"good",6000,"automatic","sedan").
car("Tesla","white",4,"good",100000,"automatic","sedan").
car("Honda","black",4,"medium",3000,"manual","suv").
car("Hyundai","grey",5,"good",8000,"automatic","suv").
car("Kia","blue",7,"good",7500,"manual","suv").
car("Ford","white",5,"medium",10000,"automatic","suv").
car("Jeep","black",5,"good",15000,"automatic","suv").
car("Nissan","silver",7,"medium",9000,"manual","suv").
car("Audi","blue",5,"good",20000,"automatic","sedan").
car("Volkswagen","white",5,"medium",12000,"manual","sedan").
car("Skoda","blue",5,"good",11000,"automatic","sedan").
car("Chevrolet","red",5,"medium",9500,"manual","sedan").
car("Volvo","grey",5,"good",18000,"automatic","sedan").

% Start the system
start :-
    writeln(" Welcome to the CarGenie 🚙 🚙 🚙 "),
    writeln(" Let's Turn Your Car Dreams Into Reality!\n\n "),

    % Ask user preferences

    write("Enter preferred type (Sedan/SUV): "),
    read_line_to_string(user_input, TypeInput),
```

```
    string_lower(TypeInput, TypePref),

    write("Enter preferred color (Black/White/Red/Blue/Silver): "),
    read_line_to_string(user_input, ColorInput),
    string_lower(ColorInput, ColorPref),

    write("Enter maximum budget: "),
    read_line_to_string(user_input, BudgetInput),
    number_string(MaxBudget, BudgetInput),

    write("Enter preferred transmission (Automatic/Manual): "),
    read_line_to_string(user_input, TransInput),
    string_lower(TransInput, TransmissionPref),

    nl,
    % Call the recommendation engine
    recommended_car(TypePref, ColorPref, MaxBudget, TransmissionPref).

% Recommendation logic
recommended_car(TypePref, ColorPref, MaxBudget, TransmissionPref) :-
    % Step 1: Exact matches for type, color, transmission
    findall(
        car(Car, Colour, Seats, Fuel, Price, Transmission, Type),
```

```
        (car(Car, Colour, Seats, Fuel, Price, Transmission, Type),
         Type = TypePref,
         Colour = ColorPref,
         Transmission = TransmissionPref),
        Matches
    ),

    (   Matches == [] ->
        writeln("No exact car found with given type, color, and transmission."),
        suggest_alternatives(TypePref, ColorPref, TransmissionPref, MaxBudget)
    ;   % Step 2: Filter by budget
        include(within_budget(MaxBudget), Matches, BudgetMatches),
        (   BudgetMatches == [] ->
            writeln("No car found in this budget. Suggesting alternatives..."),
            suggest_alternatives(TypePref, ColorPref, TransmissionPref, MaxBudget)
        ;   writeln("Cars found within budget:"),
            print_cars(BudgetMatches)
        )
    ).

% Suggest alternatives: cars matching preferences but <= 20% over budget
suggest_alternatives(TypePref, ColorPref, TransmissionPref, MaxBudget) :-
    MaxAlt is MaxBudget * 1.2,  % Allow up to 20% over budget
```

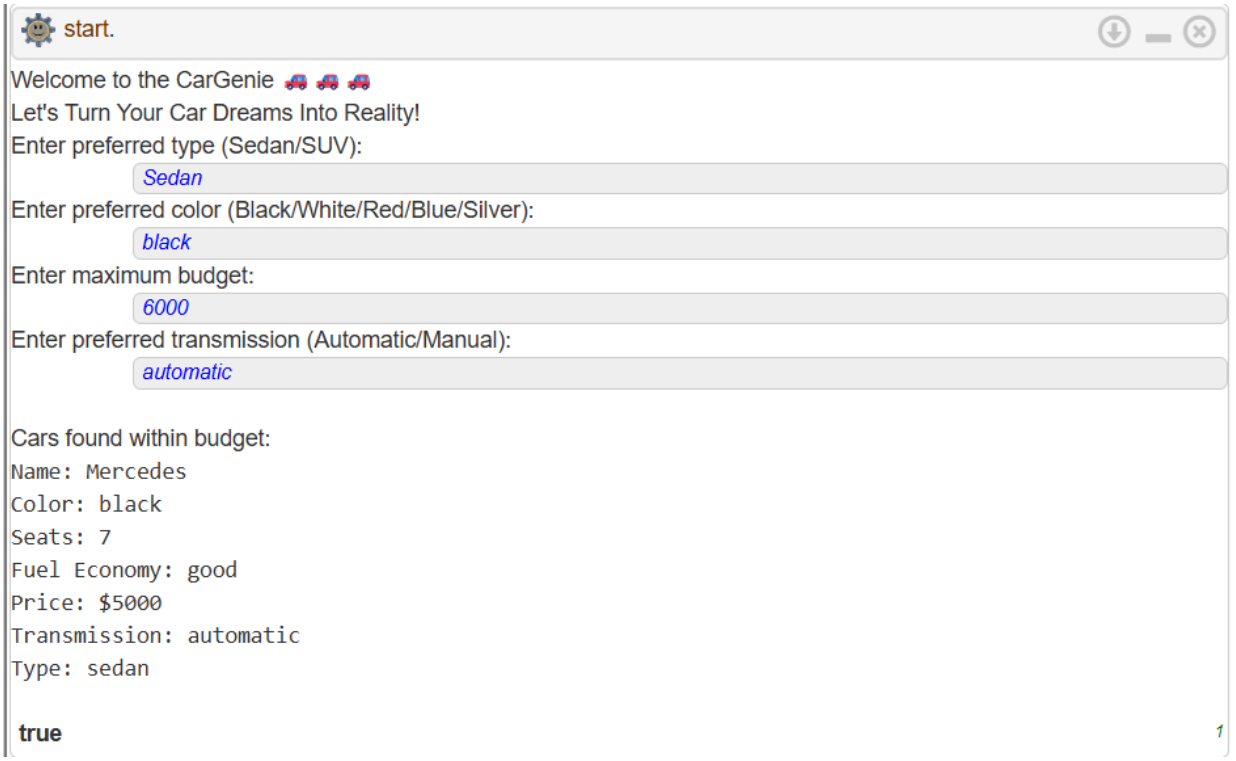
```
    findall(
        car(Car, Colour, Seats, Fuel, Price, Transmission, Type),
        (car(Car, Colour, Seats, Fuel, Price, Transmission, Type),
         Type = TypePref,
         Colour = ColorPref,
         Transmission = TransmissionPref,
         Price =< MaxAlt,
         Price > MaxBudget),
        AltMatches
    ),
    (   AltMatches == [] ->
        writeln("No close alternatives found.")
    ;   writeln("Close matches slightly above budget:"),
        print_cars(AltMatches)
    ).

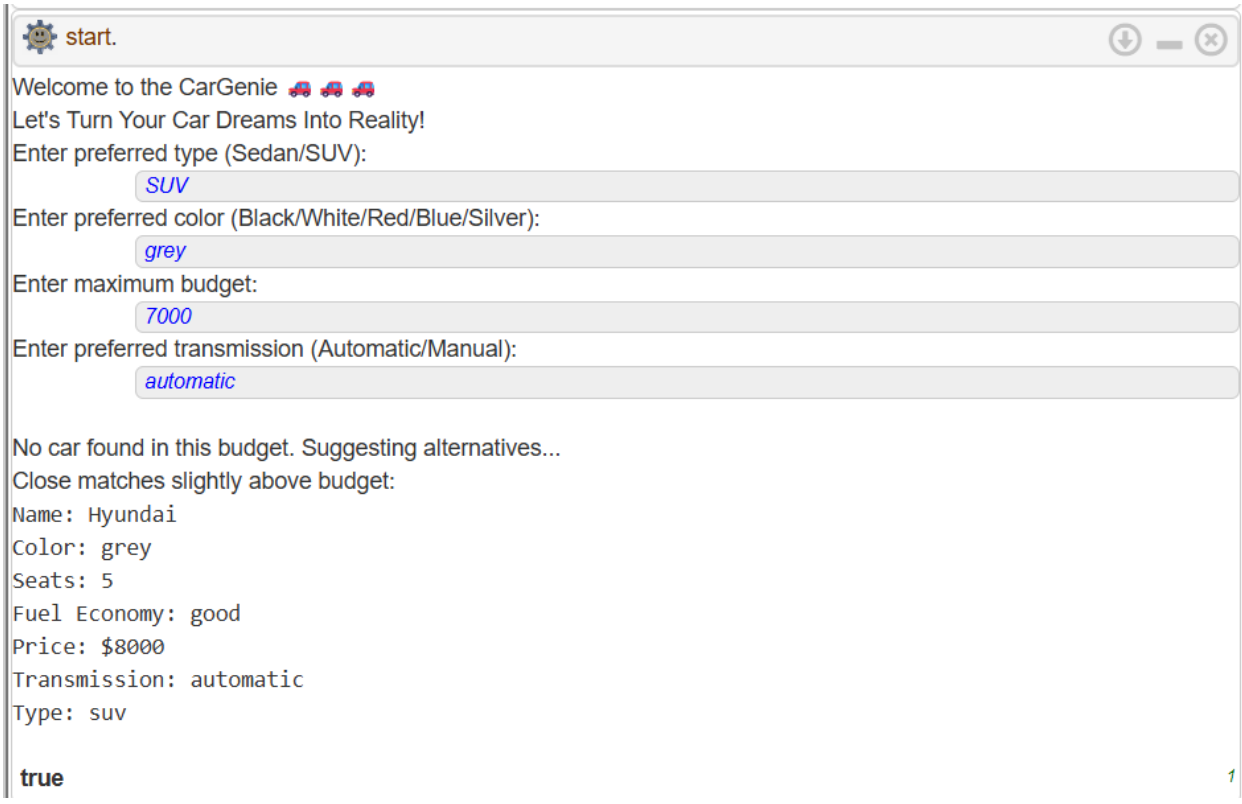

% Budget check
within_budget(Max, car(_, _, _, _, Price, _, _)) :-
    Price =< Max.
% Print car details nicely
print_cars([]).
print_cars([car(Car, Colour, Seats, Fuel, Price, Transmission, Type) | Rest]) :-
    format("Name: ~w~nColor: ~w~nSeats: ~w~nFuel Economy: ~w~nPrice:
$~w~nTransmission: ~w~nType: ~w~n~n",
```

[Car, Colour, Seats, Fuel, Price, Transmission, Type]),

print_cars(Rest).

Output:





Conclusion:

From this experiment, I learned how to design a simple expert system in Prolog. By using facts and rules, the system can recommend cars based on user preferences. If no suitable car is available within budget, it can also provide close alternatives. This shows how Prolog can be used for decision-making applications in real life.